



Department of Computer Science and Engineering
Final Examination Summer 2025
CSE 321: Operating Systems

Duration: 2 Hours

Total Marks: 50

Answer all the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Sec:
--------------	------------	-------------

1. a) Write the answer in the question paper.

[0.5

CO1 Find outputs of the given code:

*6=

3]

```

2  int direction = 0;
3  int array[] = {8,16,24};
4  int high;
5  int low;
6  void searchForIndex(int value);
7  int main(void) {
8      int count = sizeof(array) / sizeof(int);
9      high = count - 1;
10     low = 0;
11     searchForIndex(8);
12     return 0;
13 }
14 void searchForIndex(int value) {
15     while (1) {
16         printf("Current search range: low = %d, high = %d\n", low, high);
17         if (high < low) {
18             exit(0);
19         }
20         int mid = (high + low) / 2;
21         if (array[mid] == value) {
22             printf("Value %d found at index %d\n", value, mid);
23             exit(mid);
24         }
25         else {
26             pid_t pid = fork();
27             if (direction % 2 == 0) {
28                 if (pid < 0) {
29                     perror("fork failed");
30                     exit(1);
31                 }
32                 else if (pid > 0) {
33                     wait(NULL);
34                     high = mid - 1;
35                 }
36                 else {
37                     low = mid + 1;
38                 }
39             }
40             else {
41                 if (pid < 0) {
42                     perror("fork failed");
43                     exit(1);
44                 }
45                 else if (pid > 0) {
46                     wait(NULL);
47                     low = mid + 1;
48                 }
49                 else {
50                     high = mid - 1;
51                 }
52             }
53             direction = direction + 1;
54         }
55     }
56 }
57 }

```

Write the outputs in the question paper. The output sequence should be exactly matched.

Outputs	
1	
2	
3	
4	
5	
6	

b) Write the answer in the answer script.

[1*2
=2]

A programmer writes a C program that uses **malloc()** to request a large block of memory to store a data structure.

- I. **Identify** in which section of the PCB stores the data structure created from the **malloc()**?
- II. **State** how does the size of this section change during program execution compared to the text section?

2. Write the answers in the answer script. [1*2
CO2 =2]
- a) Alice is designing a "Very Simple File System" (vsfs) on a small **64 block** disk. She has allocated regions for the superblock, bitmaps, and the inode table. She wants to maximize the space available for user files.
- Identify** which region of the file system should be the largest?
 - Why** is dedicating the majority of space to this region a fundamental principle of file system design?
- b) Bob reports that he cannot create any new files on a disk partition, but a disk usage tool shows that there are still several gigabytes of free space. [1*2
=2]
- Identify** which specific region of the file system is responsible for the issue?
 - In this scenario, **explain** why having free data space does not guarantee that a new file can be created?
- c) A file system uses UNIX inode data structure which contains **12** direct block addresses, some single indirect blocks, **16** double indirect blocks, and **3** triple indirect blocks. The size of each block is **32 Bytes** and the size of each pointer is **8 Bytes**. The maximum possible file size is **20.375 KB**. [1*2
=2]
- Calculate** how many pointers can be stored in a block in this file system?
 - Calculate** the number of single indirect blocks in the system.
- d) Dipu executes an application which creates two hard links to an existing file. As a result, now there are three different pathnames that point to the same file content. [1*2
=2]
- Detect** how many inodes are used for these three pathnames?
 - What** value will be retrieved from the **links_count** field of the inode?
- e) A file system has inode size = **128 Bytes** and block size = **16 KB**. First **4 blocks** contain superblock, group descriptor, data bitmap and inode bitmap. [1+2
=3]
- Calculate** the address of the inode number **353**.
 - The disk is sector addressable and the size of each sector is **1 KB**. **Find out** the sector address of the inode block.

f) A program opens a file `/a/b/c/d.txt` and then immediately issues a `read()` call to get the first **4 KB** of data. **[1*2 =2]**

- I. **Write** whether the `read()` operation needs to modify the data bitmap or not in this scenario.
- II. **Justify** your answer of **part (I)**.

g) Journaling procedure of a file system is about appending a data block to a file. This involves updating the inode (**I[v2]**), the data bitmap (**B[v2]**), and the new data block (**Db**). **[1*2 =2]**

- I. Using data journaling, **identify** which blocks are written to the journal to form the transaction?
- II. **Write** what is the process of writing these pending updates from the journal to their final locations called?

h) Dipto is doing a large sequential write (e.g., saving a large video file). Suddenly he noticed that his disk performance is roughly half of its maximum write speed. **[1*2 =2]**

- I. **Identify** the type of journaling responsible in this scenario for the issue.
- II. **Why** does it have this effect during write heavy operations?

i) Kabbya is designing metadata journaling in a file system. In his journaling method, he considers writing the metadata transaction to the journal first, and only then writing the user's data block to its final location. **[1*2 =2]**

- I. **State** that why his preferred order can be dangerous as a journaling method?
- II. **What** specific inconsistency could arise in his journaling if a crash occurs after the journal is written but before the data block is?

j) A web server receives thousands of client requests concurrently. The server needs to handle these requests efficiently without the high overhead of creating a new process for each one. **[1*2 =2]**

- I. **Identify** what is a more economical approach than creating a new process for each request?
- II. **State** how does this approach work when a new request arrives?

k) A complex application has **four** threads running on a quad-core system. [1*2
=2]
Thread 1 reads data from the network, **Thread 2** performs a statistical analysis on it, **Thread 3** updates a database, and **Thread 4** updates the UI.

- I. **Identify** what type of parallelism is functional in the application?
- II. **State** what must the programmer ensure if **Thread 2** depends on the data retrieved by **Thread 1**?

l) Consider the classic deadlock scenario with processes P0 and P1 and semaphores S and Q. P0 executes **wait(S)**, and is then interrupted. P1 then executes **wait(Q)**. [1*2
=2]

- I. **Identify** what happens when P0 resumes and executes **wait(Q)**?
- II. **State** what happens if P1 executes **wait(S)** in the meantime?

3. **Write the answers in the answer script.**

C03 a) A user program was derived by a programmer and generally the program is a piece of code and remains in the secondary memory. In order to execute it first the program needs to be compiled and upon successful compilation an executable file is generated which is basically run by the system. When the program is compiled and loaded into the memory, the logical addresses generated by the CPU are identical to the physical addresses accessed in RAM. [1*2
=2]

- I. **Identify** specifically which stage of the address binding scheme the scenario occurs.
- II. **State** about the stage of address binding when the logical addresses differ from the physical addresses.

b) A process is running in a system in which the page size is **16384 Bytes**. **13 pages** are required to allocate the process in the system. [1*2
=2]

- I. **Calculate** the size of the process.
- II. **Calculate** the **internal fragmentation** of the process after paging.

c) In a Linux system, forty different user processes are running and all of them use the standard library of C (**libc**). However, analysis of physical memory shows only one copy of the **libc** code. [1*2
=2]

- I. **Identify** which feature of paging allows these processes to use the same library code without each having its own copy?
- II. **State** a key requirement for the code in the library to be sharable in this way.

d) In a **32-bit** system with a **two-level** page table, the logical address is split into a **7-bit outer page number**, a **9-bit inner page number**, and a **16-bit offset**. [1*2 =2]

- I. **Calculate** how many entries can the outer page table hold in this system?
- II. **State** what does each entry in the outer page table point to?

e) The MMU attempts to translate a logical address for a process, but the page table entry for the required page has its valid-invalid bit set to **"invalid"**. [1*2 =2]

- I. **Identify** what is the reason this bit might be set to **"invalid"** in a demand-paging system?
- II. **State** what event is triggered when the hardware encounters this **"invalid"** bit during the address translation?

f) A system designer is comparing **FIFO** and **Optimal** page replacement algorithms on a sample reference string. The **Optimal** algorithm performs better in this test case and provides less page faults. [1*2 =2]

- I. **Identify** what is the key difference in the information each algorithm uses to make a replacement decision?
- II. **State** how does this difference account for the better performance of the **Optimal** algorithm?

g) Consider a system with TLB hit ratio of **97.25%**, and the TLB access time of **17 ns**. If Effective Access Time is **227.6375 ns**, then **calculate** the main memory access time. [1*2 =2]

h) A system is processing the reference string **7, 0, 1, 2, 0, 3, 0, 4,** with three frames, which currently hold pages **{2, 0, 3}**. The order of previously demanded pages are **page 0, then page 3, and page 2**. The next reference is to **page 4**, requiring a replacement. [1*2 =2]

- I. If the system uses **LRU** page replacement algorithm and the CPU demands for **page 4** then find out which page should be swapped out in order to swap in the demanded page in this scenario?
- II. **Justify** why this page is being swapped out.

- i) A process runs in a system with **3-level** paging and it has a logical address space of **16-bits**. In the system page size is **16 Bytes**, size of each entry of the page table is **1 Byte** and size of the main memory is **1 KB**. After the paging the logical address space of the process looks like: **[1*4 =4]**

2 nd Outer Page #	Outer Page #	Inner Page #	Offset
P1	P2	P3	d
4	4	4	4

Necessary page table information is given below:

2 nd Outer PMT		
Page #	Frame #	v/i bit
0		0
1		0
2		0
3		0
4		0
5	31	1
6		0
7		0
8		0
9		0
10		0
11		0
12	90	1
13		0
14		0
15		0

Outer PMT						Inner PMT					
Frame 31			Frame 90			Frame 4			Frame 54		
Page #	Frame #	v/i bit	Page #	Frame #	v/i bit	Page #	Frame #	v/i bit	Page #	Frame #	v/i bit
0			0			0			0	43	1
1			1			1			1		
2			2			2			2		
3			3	87	1	3			3		
4			4			4			4		
5			5			5			5		
6	10	1	6			6	256	1	6		
7			7			7			7	101	1
8			8			8			8		
9			9			9			9		
10			10			10			10		
11			11	4	1	11			11		
12			12			12			12		
13			13			13	17	1	13		
14	54	1	14			14			14		
15			15			15			15		

In this system, if the CPU generates logical addresses I) **52078** and II) **24179** then map the corresponding physical addresses of these logical addresses. If the physical address is not valid then write **“invalid”**.